

Rationale: Diffusion Noise Schedule Visualizer

Web Link: https://y1hnnn.github.io/Diffusion_Noise-Schedule-Visualizer/

Diffusion schedules are usually explained through equations and curves, but it's hard to connect those curves to what actually happens to an image. The **Diffusion Noise Schedule Visualizer** makes that connection tangible by running three schedules — Linear, Cosine, Exponential — in lockstep on the same image and noise.

The page splits into Forward Noising (closed-form corruption) and Denoising (a real pretrained DDPM recovering from a partially-noised input).

Design Choice

One intentional design choice was to include two comparison modes.

- **Same timestep mode** evaluates all schedules at the same t — the same step produces visibly different signal retention.
- **Same signal retention mode** slides a target $\bar{\alpha}$. Each schedule inverts its $\bar{\alpha}_t$ curve to find the t^* where it matches, so the noise level is identical and the timestep differs.

The dual lens reveals two distinct effects of schedule choice: how fast the signal degrades at fixed t , and how the model's t -conditioning differs at fixed noise.

Another choice is that feedback is both qualitative and quantitative.

- Image panels make schedule differences immediately visible
- Overview plots (signal retention, noise amount, SNR) show the corresponding curves
- Final recovery is scored with MSE, PSNR, and SSIM, with a best-overall badge surfacing the schedule that wins the most metrics.

Design Iteration

The first version was a lightweight in-browser simulator: forward noising rendered live, denoising approximated by a JavaScript oracle that cheated with the source image. This visualized schedule shape, but didn't reflect a real model.

I then moved inference to a Python script that runs a pretrained DDPM and pre-renders denoising trajectories per (image, seed, schedule, strength, target- $\bar{\alpha}$); the frontend loads PNGs and stays responsive.

Demo Instruction

Load Pre-rendered Images

Download images/ and results/ from [Google Drive](#) into the project root (alongside index.html and generate_cache.py). The frontend loads image options, denoising frames, and Final Quality metrics directly from those caches.

The project structure should look like this:

```
project-root/
├── index.html
├── manifest.json
├── generate_cache.py
├── images/
│   ├── ...
└── results/
    ├── ...
```

Add Your Own Images

Drop files into images/ and run from the project root:

```
python generate_cache.py \
  --target-alphas 0.8 0.6 0.4 0.2 0.1 0.05 0.01 0.001 \
  --strengths 0.3 0.45 0.6 0.75 0.9 \
  --seeds 0 1 2 3 4
```

--strengths drives Same timestep

--target-alphas drives Same signal retention;

Multiple --seeds give the Regenerate-Noise button distinct trajectories to cycle through.